

Connect Odoo (ERP/inventory) ↔ Skipily (step by step)

Prerequisite: First read `00-Skipily-Integration-Basics.md` and have your **Skipily API key** ready.

Why Odoo? Odoo is one of the world's most widely used ERP/inventory systems for small and medium businesses – it combines warehouse, sales, invoicing and shipping in one tool and integrates very well.

Important: The steps shown here also work the same way with other ERP systems such as **JTL-Wawi**, **Xentral**, **Microsoft Dynamics 365 Business Central** or **SAP Business One** – everywhere it's the same pattern: *products out* → *orders in* → *shipment back*.

We connect everything via the free no-code tool **Make.com**. So you need neither an ERP developer nor server access.

 **Shortcut – ready-made template:** A finished Make template "Odoo ↔ Skipily" does not exist in the public gallery (yet), but we provide **importable blueprints** (folder `blueprints/`) that pre-fill the entire Skipily part (URL, headers, JSON): `skipily-create-product.json` (Part A), `skipily-receive-order.json` (Part B), `skipily-report-shipment.json` (Part C). **Import:** New scenario → bottom **"..." (More)** → **"Import Blueprint"** → choose file. After that you only need to insert your `x-api-key`, add the Odoo module and map the fields – exactly what Parts A–C show. The blueprints work unchanged with JTL, Xentral, Dynamics or SAP B1 as well.

What works in the end:

- New/changed item in Odoo → automatically appears on Skipily.
- Order on Skipily → automatically created as a sales order in Odoo.
- Delivery/shipment confirmed in Odoo → status + tracking number go back to Skipily.

Allow **about 60 minutes** (ERP has a few more fields).

Part 0 – Preparation (15 min)

1. **Create a Make.com account:** <https://www.make.com> → "Get started free".
2. Have your **Skipily API key** ready (see Basics).
3. Prepare **Odoo access for Make**. Make connects to your Odoo instance via:
 - **Server address** of your Odoo (e.g. `https://yourbusiness.odoo.com`)
 - **Database name** (for Odoo Online it's usually shown in the login area; ask Odoo support if needed)

- **Username** (your Odoo login email)
- **API key** from Odoo: in Odoo top right click your name → "**My Profile**" → tab "**Account Security**" → "**New API Key**" → name `Skipily` → **copy** and store safely.

Not using Odoo? Then you need the same trio from your ERP provider: **server address, user, API key/token**. The rest of the guide stays the same – only the module names in Make will read e.g. "JTL" or "Dynamics".

Now you have: Skipily API key + Odoo access (server, DB, user, API key).

Part A – Send items from Odoo to Skipily (20 min)

Goal: New/changed item in Odoo → automatically on Skipily.

1. In **Make.com** → "**Create a new scenario**".
2. "+" → search "**Odoo ERP**" → module "**Watch Records**".
 - "**Add**" at Connection → enter server address, database, user, API key from Part 0 → **Save**.
 - **Model / object**: `product.template` (the "item master" in Odoo).
 - **Trigger**: "Created or updated".
3. Next to it click "+" → "**HTTP**" → "**Make a request**" (the Skipily call):
 - **URL**: `https://vcjwlyqkfkszumdrfvtn.supabase.co/functions/v1/products-api`
 - **Method**: `POST`
 - **Headers**:
 - `Authorization` → `Bearer eyJhbGci...` (public value from the Basics)
 - `x-api-key` → **your Skipily API key**
 - `Content-Type` → `application/json`
 - **Body type**: `Raw` → **JSON (application/json)**
 - **Request content** (insert Odoo fields by clicking):

```
{
  "name": "{{name from Odoo}}",
  "sku": "{{default_code (internal reference) from Odoo}}",
  "manufacturer": "{{manufacturer/brand, if maintained}}",
  "price": "{{list_price from Odoo}}",
  "currency": "EUR",
  "stock_quantity": "{{qty_available from Odoo}}",
  "in_stock": true,
  "weight_kg": "{{weight from Odoo}}",
  "ean": "{{barcode from Odoo}}",
  "description": "{{description_sale from Odoo}}"
}
```

4. **Save** → **"Run once"** → create/change a test item in Odoo → scenario runs green → item appears on Skipily.
5. Bottom left, turn **"Scheduling" ON** (e.g. every 15 min).

Clean updating (recommended for ERP): Add a custom field "Skipily ID" per item in Odoo and write the `id` returned by Skipily there. On the next run: if the Skipily ID exists → **PUT** `.../products-api?id=THE-ID` (update); otherwise **POST** (new). In Make you handle this with a **Router** + filter "Skipily ID is empty?". For the start, plain creation is enough.

Part B – Create Skipily orders as sales orders in Odoo (15 min)

Goal: purchase via Skipily → automatically a **sales order (sale.order)** in Odoo. Skipily "rings" via **webhook**.

1. **New scenario** in Make.
2. First module **"Webhooks"** → **"Custom webhook"** → **"Add"** → name `Skipily orders` → **copy the URL**.
3. Store the URL in Skipily: **Provider portal** → **Settings** → **Integrations** → **"Webhook URL for orders"** → paste → save. (No field? URL to support@skipily.app.)
4. Second module **"Odoo ERP"** → **"Create a Record"**:
 - **Model:** `sale.order` (sales order). Map customer, shipping address, positions from the Skipily webhook. Items are matched to Odoo items via the **SKU** (`default_code`) – if needed add a "Search Records" module beforehand that finds the Odoo item ID for the SKU.
5. **Save + Scheduling ON**.
6. **Test:** test order on Skipily → sales order appears in Odoo.

Remember the Skipily order ID (`order_id` / `order_number` from the webhook) → write it into an Odoo field on the order (e.g. "Customer Reference"/ `client_order_ref`). You need it in Part C.

Part C – Report delivery & tracking back to Skipily (15 min)

Goal: When you confirm delivery in Odoo (delivery order "done", tracking number recorded), the customer sees "shipped" + tracking number on Skipily.

1. **New scenario** in Make.
2. First module **"Odoo ERP"** → **"Watch Records"**:
 - **Model:** `stock.picking` (deliveries/delivery orders).
 - Filter: only with **status "done"** and an existing **tracking number** (`carrier_tracking_ref`).
3. Second module **"HTTP"** → **"Make a request"**:
 - **URL:** `https://vcjwlyqkfkszumdrfvtn.supabase.co/functions/v1/orders-api?id={{Skipily order ID}}` (the ID you stored on the order in Part B)
 - **Method:** `PUT`
 - **Headers:** the same three as in Part A.
 - **Body** → **Raw** → **JSON:**

```
{
  "status": "shipped",
  "tracking_number": "{{carrier_tracking_ref from Odoo}}",
  "tracking_url": "{{tracking link, if available}}"
}
```

4. **Save, Run once** (set a delivery to "done"), then **Scheduling ON**.

Done! Quick check

- ☐ Test item in Odoo → appears on Skipily (Part A)
- ☐ Test order on Skipily → sales order in Odoo (Part B)
- ☐ Delivery in Odoo "done" → Skipily shows "shipped" + tracking (Part C)

If something goes wrong

- **Make can't find the Odoo connection:** check the database name exactly (case-sensitive); regenerate the API key and re-enter it.
- **"401/403" at Skipily:** `x-api-key` or `Authorization` value wrong → copy exactly again.
- **"400 – name required":** the `name` field in the body is empty → re-map Odoo `name` .
- **Order positions empty:** the **SKU** in Odoo (`default_code`) must match the SKU from the Skipily order → add "Search Records" by SKU beforehand.
- **Other ERP (JTL/Xentral/Dynamics/SAP):** in Make replace the "Odoo" modules with those of your system; the Skipily side (URLs, headers, JSON) stays **exactly the same**.
- **Help:** support@skipily.app (ideally with a screenshot of the Make error).